

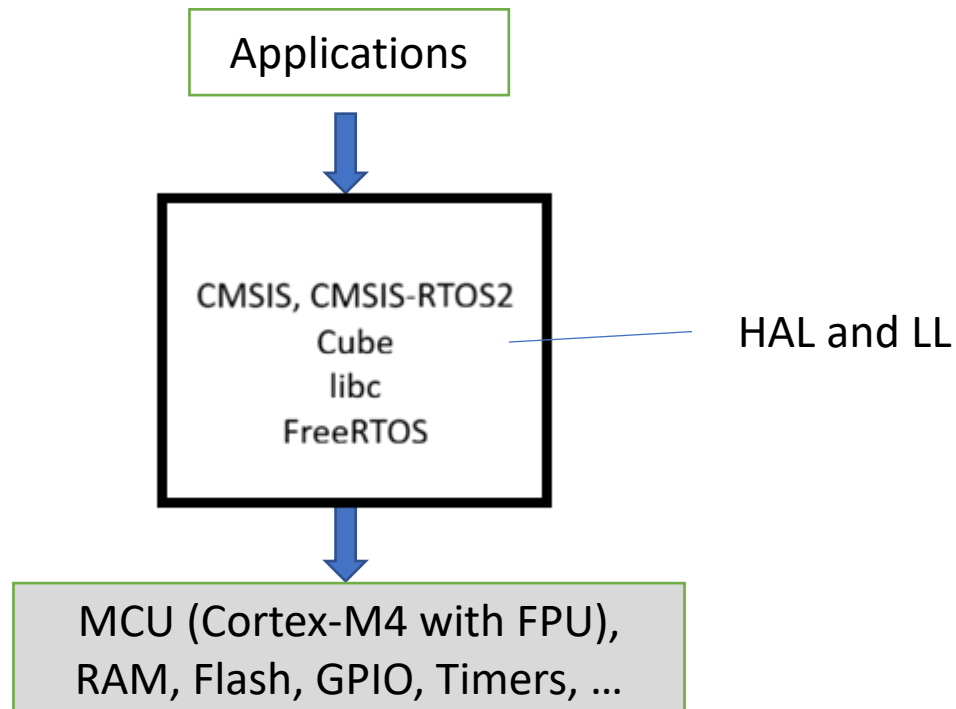
Lab: STM32CubeIDE

Introduction

Interrupt, LED blinking, FreeRTOS threads and the semaphore mechanism

Software layers

- **STM32 Cube**: Software framework with “hardware abstraction layer” (HAL) and “low layer” (LL). Cube also include the CMSIS layer.
 - Using Tools: CubeIDE and CubeMX



Overview and Download

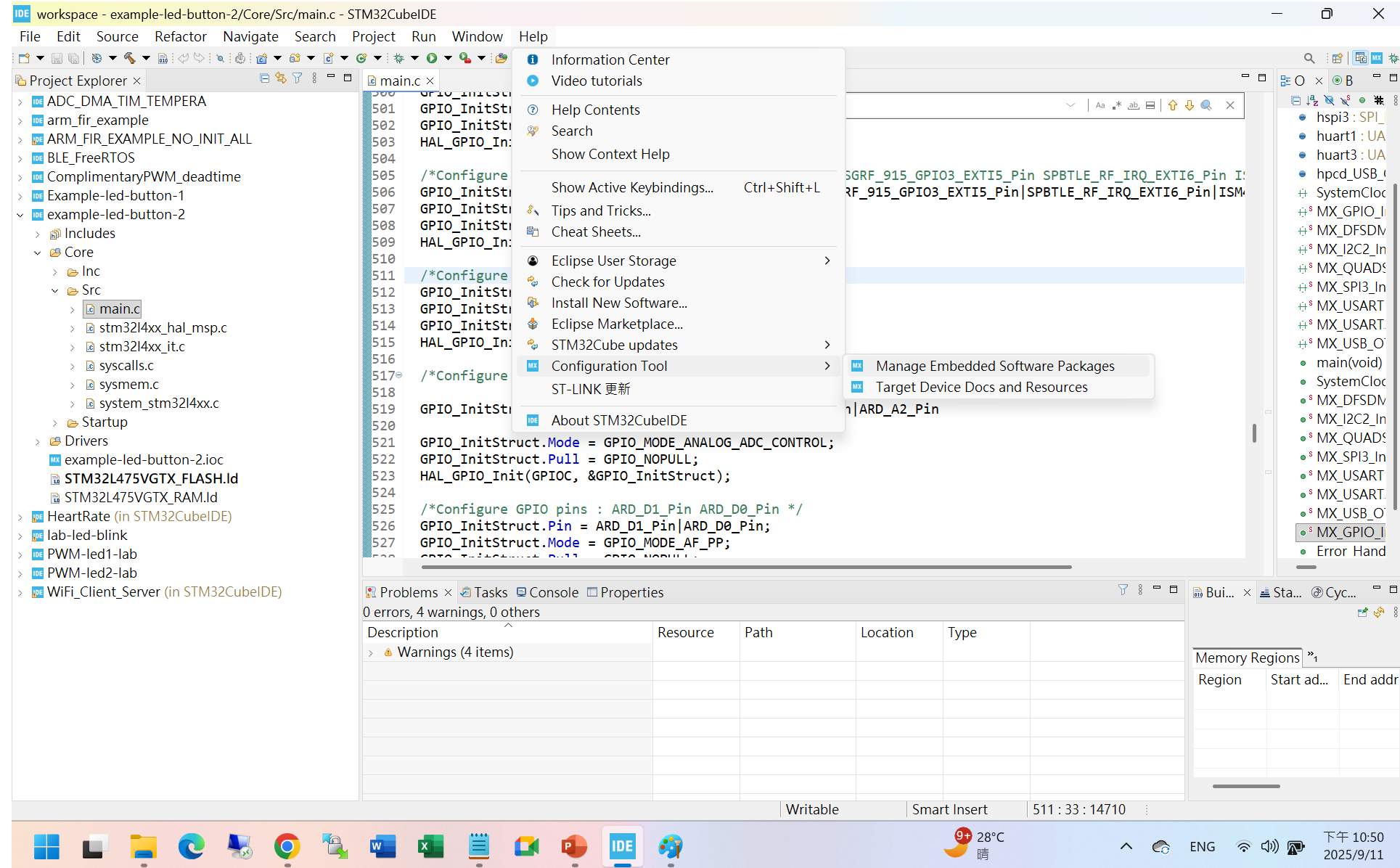
- Official web site: STM32CubeIDE
 - <https://www.st.com/en/development-tools/stm32cubeide.html>
- Download(Please register an account and install)

Software name	ECCN	OS	Version	Action
STM32CubeIDE	NEC (EU) EAR99 (US)	Windows 	2.2.0 Latest 	 Download
STM32CubeIDE for Visual Studio Code	NEC (EU) EAR99 (US)	–	–	 Go to Visual Studio Code marketplace

STMicroelectronics recommends always keeping your software up to date

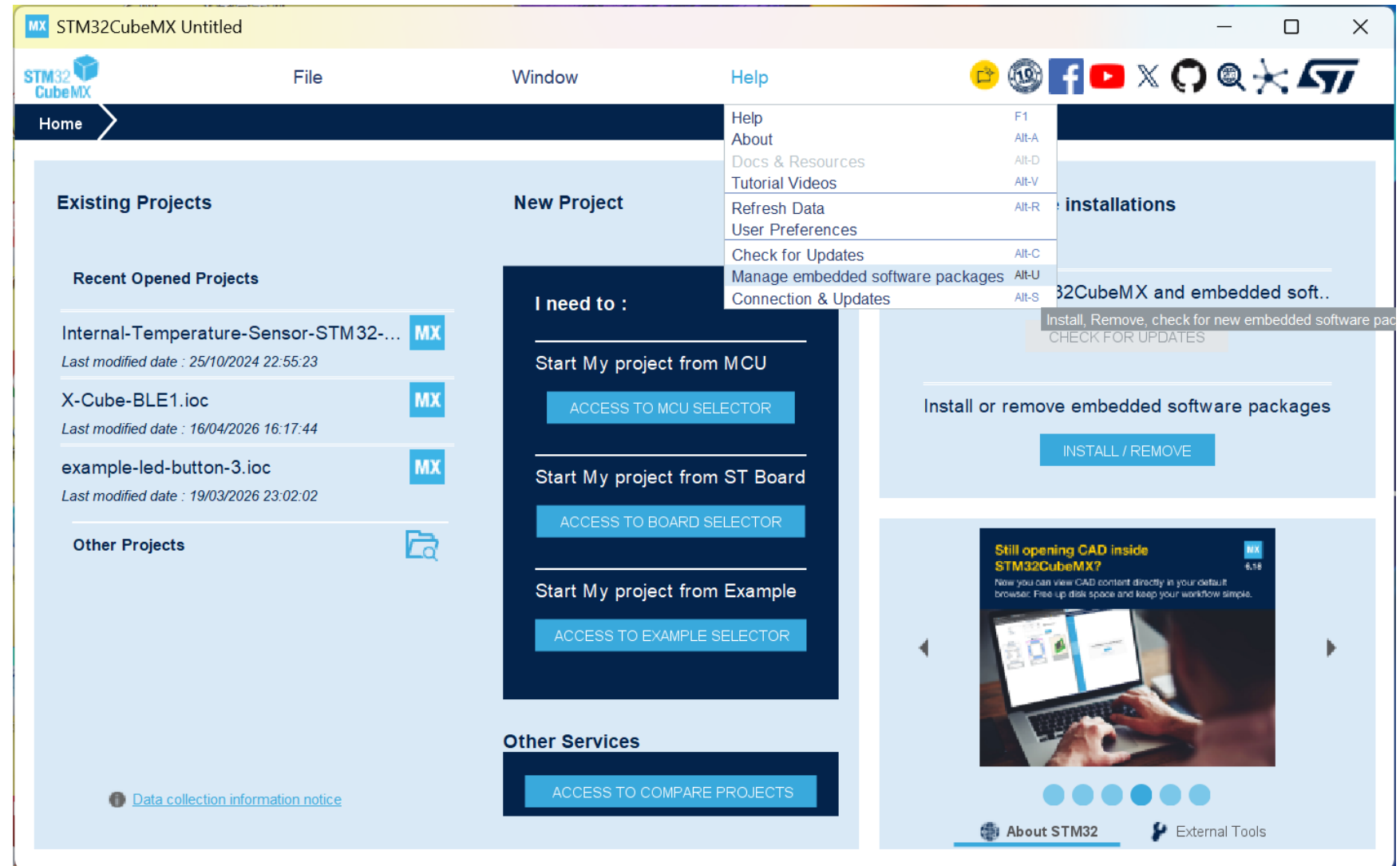
Add firmware (STM32CubeIDE 1.x)

- Manage Embedded Software Packages

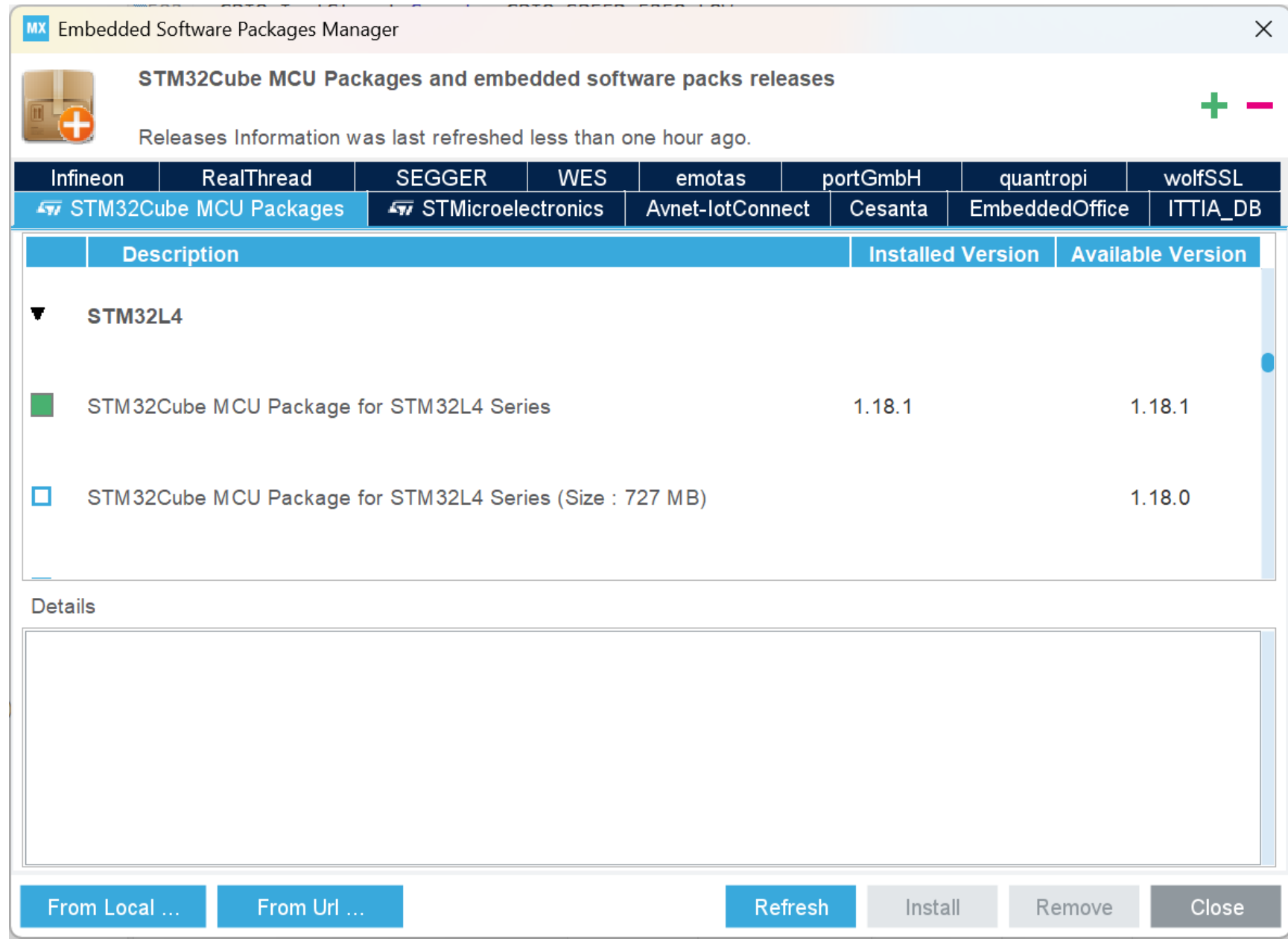


Add firmware (using STM32CubeMX)

- “Manage Embedded Software Packages” has been moved to STM32CubeMX from STM32CubeIDE 2.x



- Install STM32Cube MCU Package for STM32L4 Series : **STM32CubeL4**



Start new STM32 Project and Select MCU and Board

The screenshot displays the STM32CubeIDE MCU/MPU Selector interface. The 'Commercial Part Number' field is highlighted with a yellow circle and contains the text 'STM32L475VGT6'. The interface shows various filters on the left and a list of MCUs/MPUs at the bottom.

MCU/MPU Filters

- Commercial Part Number: STM32L475VGT6
- PRODUCT INFO
 - Segment
 - Series
 - Line
 - Marketing Status
 - Price
 - Package
 - Core
 - Coprocessor
- MEMORY
 - Flash = 1024 (kBytes)

MCUs/MPUs List: 1 item

Com...	Part No	Referen...	Mark...	Unit Pri...	Board	Package	Fla...	RAM	Fr...
☆ STM32L...	STM3...	STM32...	Active	6.3301	B-L475E-I...	B-L475E-IQ...	LQFP 100 14x...	1024... 128 ... 82 80 ...	

Target Selection

Select STM32 target or STM32Cube example

MCU/MPU Selector Board Selector Example Selector Cross Selector

Board Filters



Commercial Part Number B-L475E-IOT01A2

 + -

Marketing Status

Price >

MEMORY

Ext. Flash = 64 (MBit)

64

Ext. EEPROM = 0 (kBytes)

0

Ext. RAM = 0 (MBit)

0

FEATURES

Embedded Sensor >

User Button >

Camera >

Features

Large Picture

Docs & Resources

Datasheet

Buy

STM32L4 Series

B-L475E-IOT01A2

ACTIVE

Product is in mass production

STM32L4 Discovery kit IoT node, low-power wireless, BLE, NFC, SubGHz, Wi-Fi

Part Number : B-L475E-IOT01A

Commercial Part Number : B-L475E-IOT01A2

Unit Price (US\$) : **53.0**Mounted Device : [STM32L475VGT6](#)

The B-L475E-IOT01A Discovery kit for IoT node allows users to develop applications with direct connection to cloud servers. The Discovery kit enables a wide diversity of applications by exploiting low-power communication, multiway sensing and Arm® Cortex® -M4 core-based STM32L4 Series features.

Boards List: 1 item

Export

*	Overview	Commercial Part No	Type	Marketing Status	Unit Price (US\$)	Mounted Device
		B-L475E-IOT01A2	Discovery Kit	Active	53.0	STM32L475VGT6

< Back

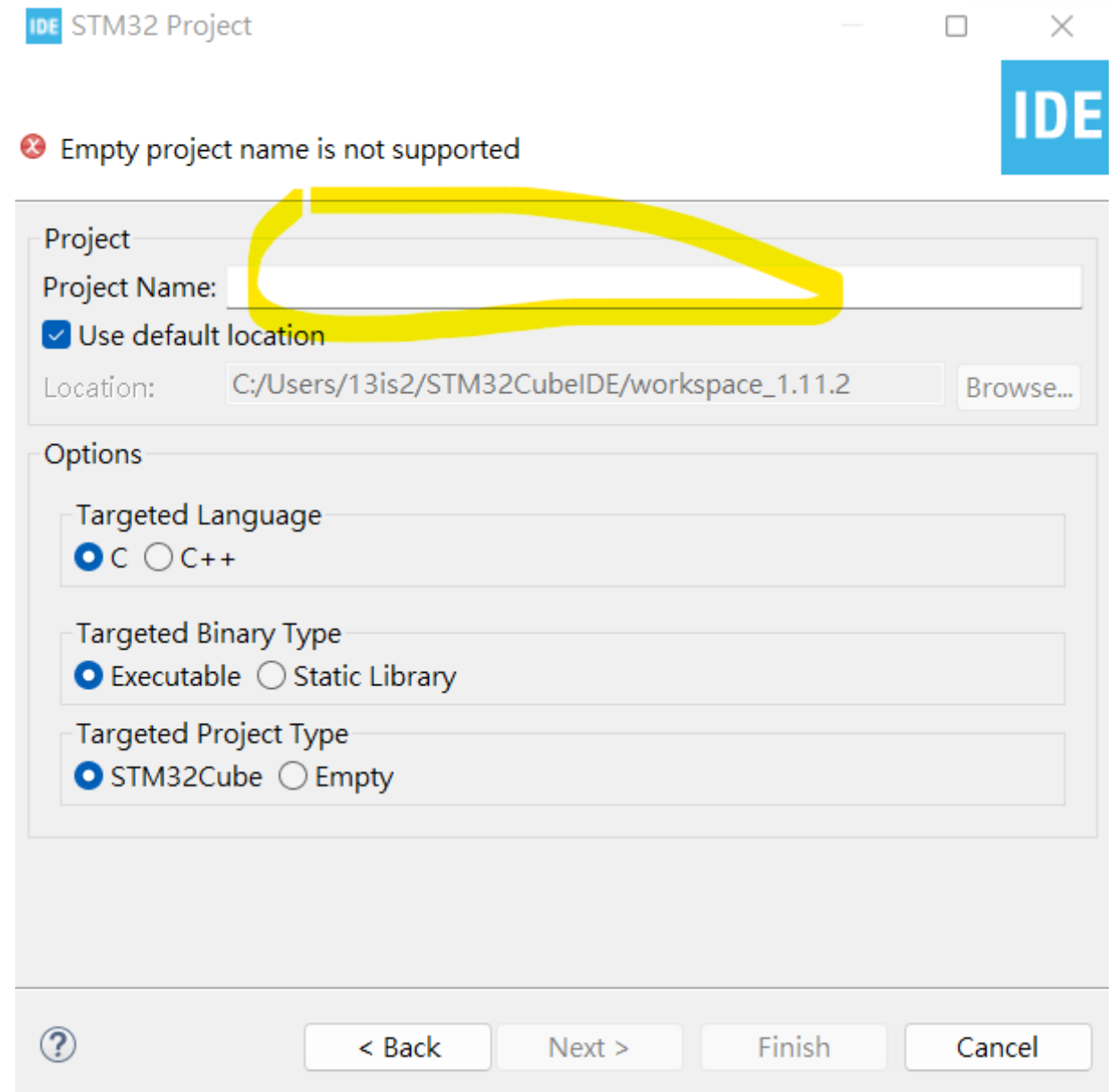
Next >

Finish

Cancel

Start new project

- STM32CubeIDE 1.x



IDE STM32 Project

Firmware Library Package Setup

Setup STM32 target's firmware

Target and Firmware Package

Target Reference: B-L475E-IOT01A1

Firmware Package Name and Version: STM32Cube FW_L4 V1.18.1

Firmware and Software Package Repository

Location:
C:\Users\sdwan\STM32Cube\Repository

See ['Firmware Updater'](#) for settings related to package installation

Code Generator Options

- ☐ Add necessary library files as reference in the toolchain project configuration file
- ☐ Copy all used libraries into the project folder
- ☒ Copy only the necessary library files

? < Back Next > Finish Cancel

IDE Board Project Options:

Initialize all peripherals with their default Mode ?

Yes No

Start New Project using STM32CubeMX

MX STM32CubeMX Untitled*: STM32L475VGtx B-L475E-IOT01A1

File Window Help

Home STM32L475VGtx - B-L475E-IOT01A1 Untitled - Project Manager GENERATE CODE

Pinout & Configuration Clock Configuration Project Manager Tools

Project

Project Name lab1-led-button

Project Location C:\Users\sdwan\STM32CubeIDE\workspace-2 Browse

Application Structure Advanced ☐ Do not generate the main()

Toolchain Folder Location C:\Users\sdwan\STM32CubeIDE\workspace-2\lab1-led-button\

Toolchain / IDE STM32CubeIDE ☒ Generate Under Root

Linker Settings

Minimum Heap Size 0x200

Minimum Stack Size 0x400

Thread-safe Settings

Cortex-M4NS

☐ Enable multi-threaded support

Thread-safe Locking Strategy Default - Mapping suitable strategy depending on RTOS selection.

Mcu and Firmware Package

Mcu Reference STM32L475VGtx

Firmware Package Name and Version STM32Cube FW_L4 V1.18.2 ☒ Use latest available version

☒ Use Default Firmware Location

Firmware Relative Path C:\Users\sdwan\STM32Cube\Repository\STM32Cube_FW_L4_V1.18.2 Browse

Pinout & Configuration

Clock Configuration

Project Manager

Tools

▼ Software Packs

▼ Pinout

 Pinout view

 System view

Q

Categories A->Z

System Core

Analog

Timers

Connectivity

Multimedia

Security

Computing

Middleware and Software Pa...

```

M24SR64_Y_RF_DISABLE [M24SR64_RFDIS]
USB_OTG_FS_OVRCCR_EXTI3 [STMPS2141STR_FAULT]
M24SR64_Y_GPO [M24SR64_GPO]
SPSGRF_915_GPIO3_EXTI5 [SPSGRF_GPIO_3]
SPBTLE_RF_IRQ_EXTI6 [BT module_SPI_IRQ]

```

BUTTON_EXTI13 [B2]
 RCC_OSC32_IN
 RCC_OSC32_OUT

ARD_A5 [ADC]
ARD_A4 [ADC]
ARD_A3 [ADC]
ARD_A2 [ADC]

ARD_D1 [UART4_TX]
ARD_D0 [UART4_RX]
ARD_D10 [SPI_SSN]

STM32L475VGTX
LQFP100

Pin	Signal	Pin	Signal
1	PA3	51	PC15
2	VSS	52	PC14
3	VDD	53	PC13
4	PA4	54	PC12
5	PA5	55	PC11
6	PA6	56	PC10
7	SP11_MISO	57	PC9
8	SP11_MOSI	58	PC8
9	ID_A1 [ADC]	59	PC7
10	ID_A0 [ADC]	60	PC6
11	INT_EXT[IO]	61	PC5
12	[ADC1_IN6]	62	PC4
13	ARD_D8	63	PC3
14	T01_DOUT	64	PC2
15	362_RSTN	65	PC1
16	PE8	66	PC0
17	DT01_CLK	67	PC15
18	435F_SCLK	68	PC14
19	435F_SCLK	69	PC13
20	435F_SCLK	70	PC12
21	6435F_I00	71	PC11
22	6435F_I01	72	PC10
23	6435F_I02	73	PC9
24	6435F_I03	74	PC8
25	33L0X_SCL	75	PC7
26	33L0X_SDA	76	PC6
27	VDD	77	PC5
28	VSS	78	PC4
29	ARD_D9	79	PC3
30	ARD_D10	80	PC2
31	ARD_D11	81	PC1
32	ARD_D12	82	PC15
33	ARD_D13	83	PC14
34	ARD_D14	84	PC13
35	ARD_D15	85	PC12
36	ARD_D16	86	PC11
37	ARD_D17	87	PC10
38	ARD_D18	88	PC9
39	ARD_D19	89	PC8
40	ARD_D20	90	PC7
41	ARD_D21	91	PC6
42	ARD_D22	92	PC5
43	ARD_D23	93	PC4
44	ARD_D24	94	PC3
45	ARD_D25	95	PC2
46	ARD_D26	96	PC1
47	ARD_D27	97	PC15
48	ARD_D28	98	PC14
49	ARD_D29	99	PC13
50	ARD_D30	100	PC12

PA13	SYS_JTMS_SWDIO
PA12	USB_OTG_FS_DP
PA11	USB_OTG_FS_DM
PA10	USB_OTG_FS_ID
PA9	USB_OTG_FS_VBUS [STMP52141STR_OUT]
PA8	SPBTLE_RF_RST
PC9	LED3_WIFI_LED4_BLE
PC8	LSM3MDL_DRDY_EXTI8 [LIS3MDL_DRDY]
PC7	VL53L0X_GPIOI0_EXTI7 [VL53L0X_GPIOI]
PC6	VL53L0X_XSHUT [Read_XSHUT]
PD15	HTS221_DRDY_EXTI15 [HTS221_DRDY]
PD14	ARD_D2 [INT0_EXTI14]
PD13	SPBTLE_RF_SPI3_CSN [BT module_SPI_CS]
PD12	USB_OTG_FS_PWR_EN [STMP52141STR_EN]
PD11	LSM6DSL_INT1_EXTI11 [LSM6DSL_INT1]
PD10	LPS22HB_INT_DRDY_EXTI0 [LPS22HB_INT_DRDY]
PD9	INTERNAL_UART3_RX [ISM43362_TX]
PD8	INTERNAL_UART3_TX [ISM43362_RX]
PB15	SPSGRF_915_SDN [SPSGRF_SDN]
PB14	LED2 [LED_GREEN]
PB13	ISM43362_WAKEUP [ISM43362_WKUP]
PB12	ISM43362_BOOT0 [ISM43362_BOOT]

STM32L475VGTx

LQFP100

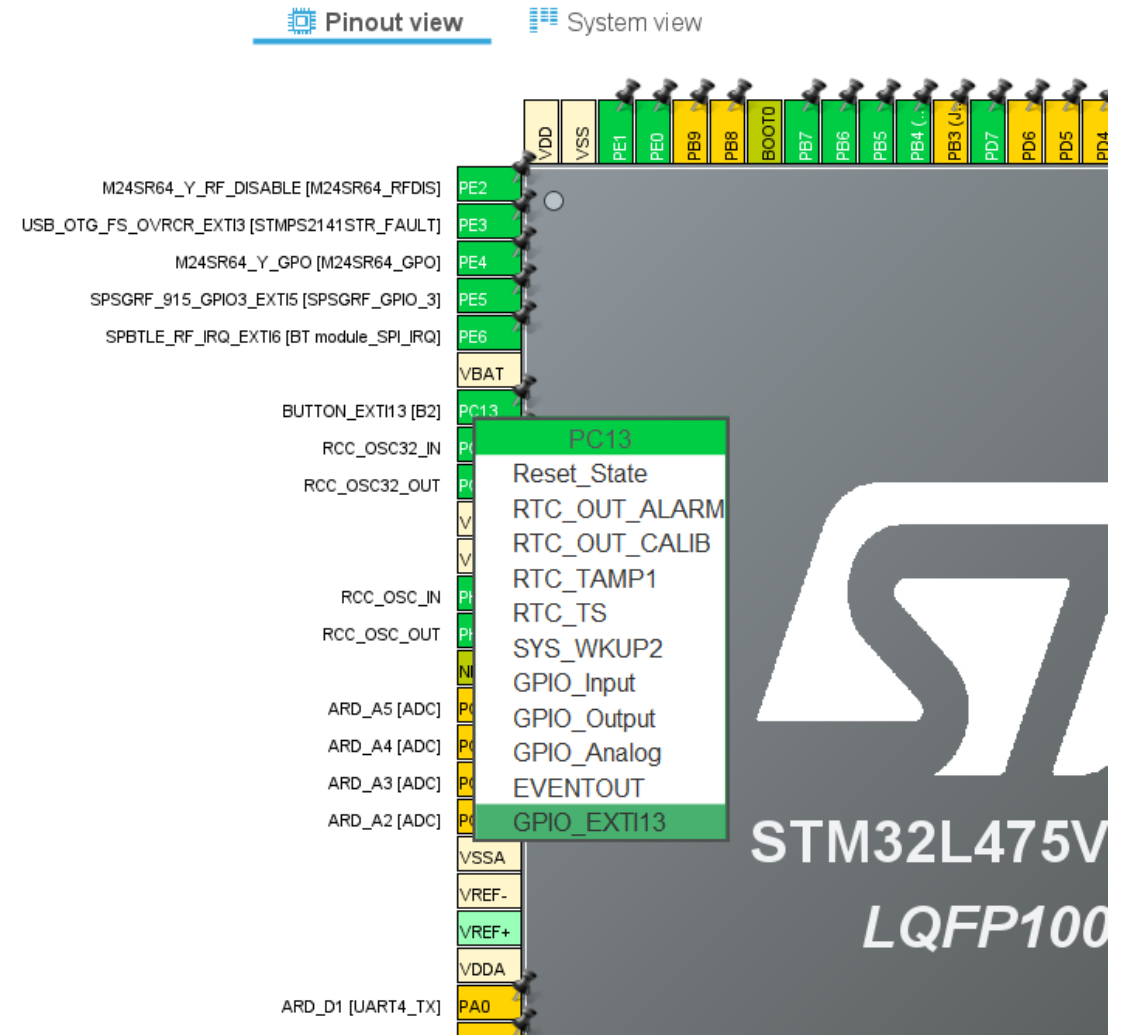
Zooming In

pb14

Unused GPIOs: 2 / 82

Example 1 : Trigger LED on by push the USER Button via interrupt

- In our target board
 - USER Button : PC13
(see B-L475E-IOT01A user manual p. 37)
- To trigger interrupt,
 - PC13 is set as GPIO_EXTI13
- In our target board
LED2: PB14
(see B-L475E-IOT01A user manual p. 29)



- > IDE ADC_DMA_TIM_TEMP
- > IDE arm_fir_example
- > IDE ARM_FIR_EXAMPLE_N
- > IDE BLE_FreeRTOS
- > IDE ComplimentaryPWM
- > IDE Example-led-button-1
- > IDE example-led-button-2
 - > Includes
 - > Core
 - > Inc
 - > Src
 - > main.c
 - > stm32l4xx_ha
 - > stm32l4xx_it.c
 - > syscalls.c
 - > system.c
 - > system_stm32
 - > Startup
 - > Drivers
 - example-led-butto
 - STM32L475VGTX
 - STM32L475VGTX_R
- > IDE example-led-button-3
 - > Includes
 - > Core
 - > Drivers
 - example-led-butto
 - STM32L475VGTX
 - STM32L475VGTX_R
 - > HeartRate (in STM32C
 - > lab-led-blink
 - > PWM-led1-lab
 - > PWM-led2-lab
 - > WiFi_Client_Server (in

example-led-button-3.ioc - Pinout & Configuration

Pinout & Configuration

Clock Configuration

Project Manager

Tools

Software Packs

Pinout

Categories

A->Z

System Core

DMA

✓ GPIO

IWDG

✓ NVIC

⚠ RCC

⚠ SYS

⊗ TSC

WWDG

Analog

Timers

Connectivity

Multimedia

Security

Computing

GPIO Mode and Configuration

Mode

Configuration

Group By Peripherals

✓ Single Mapped Signals

✓ NVIC

✓ GPIO

Search Signals

☐ Show only Modified Pins

...	Sig...	G...	GP...	G...	Ma...	Fa...	Us...	Mo...
PC...	n/a	n/a	Ext...	No...	n/a	n/a		✓
PD0	n/a	Low	Ou...	No...	Low	n/a	P...	✓
PD2	n/a	n/a	Ext...	No...	n/a	n/a	P...	✓
PD7	n/a	Low	Ou...	No...	Low	n/a	ST...	✓
PD...	n/a	n/a	Ext...	No...	n/a	n/a	LP...	✓
PD	n/a	n/a	Fxt	No	n/a	n/a	I S	✓

GPIO mode

External Interrupt Mod

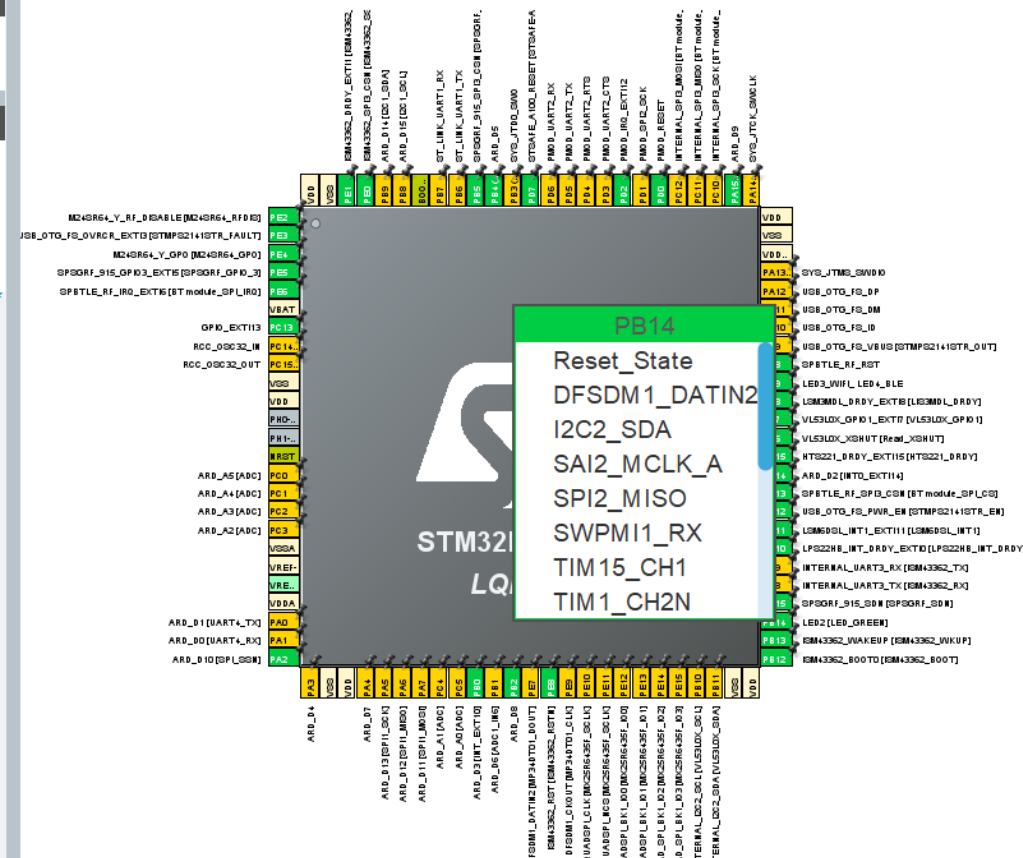
GPIO Pull-up/Pull-down

No pull-up and no pull-d

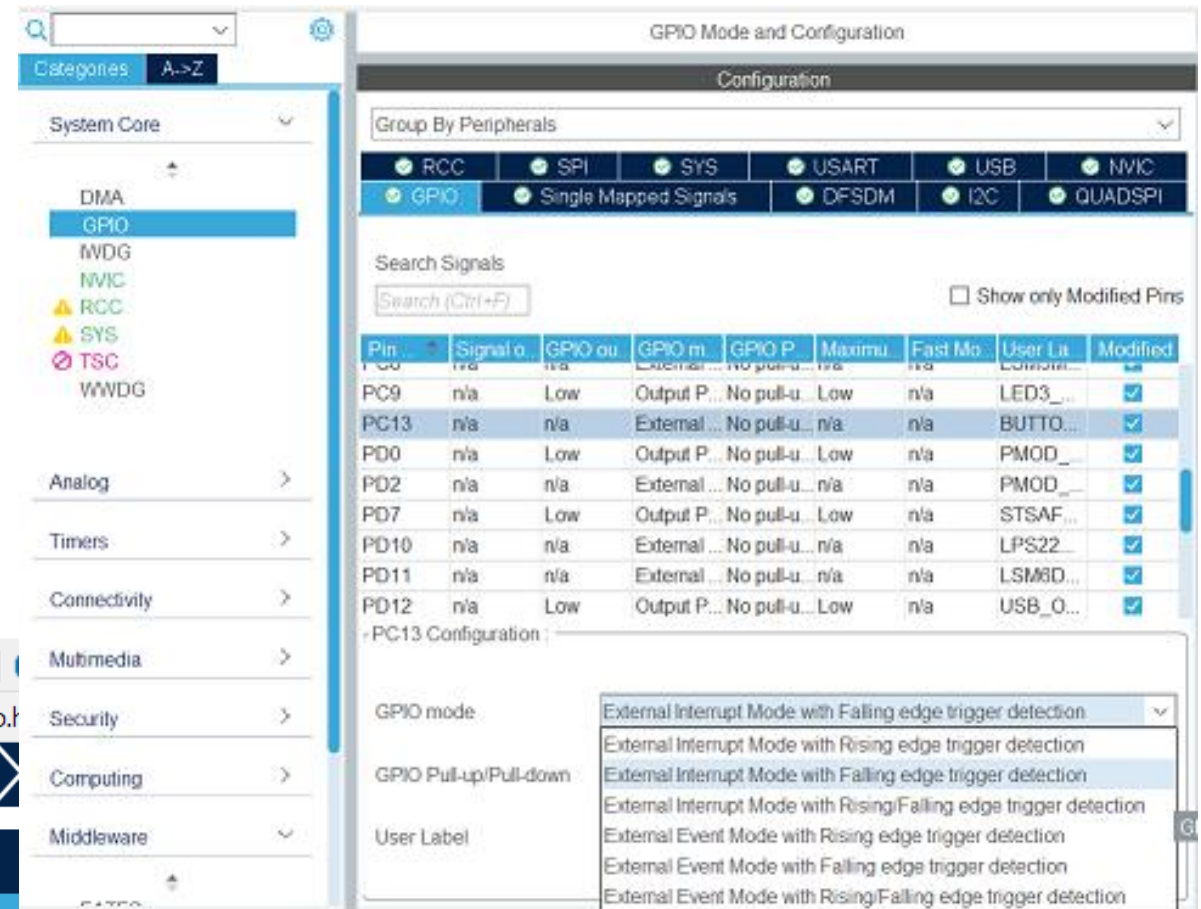
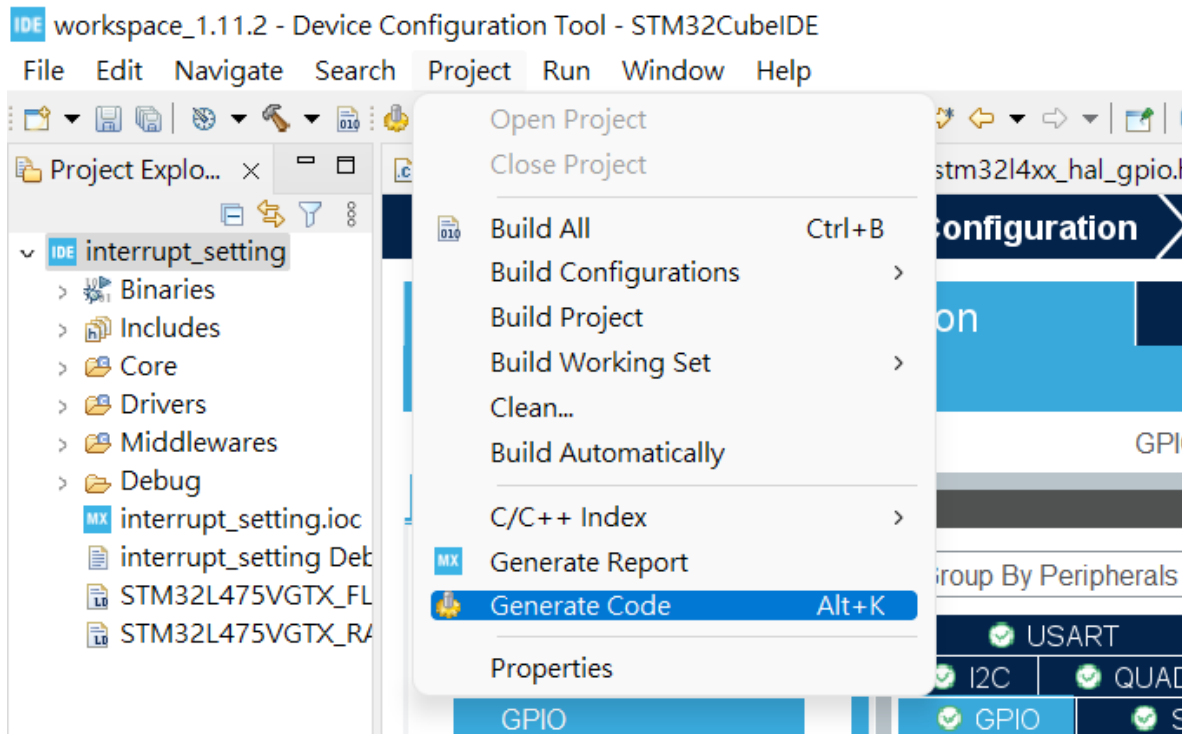
User Label

Pinout view

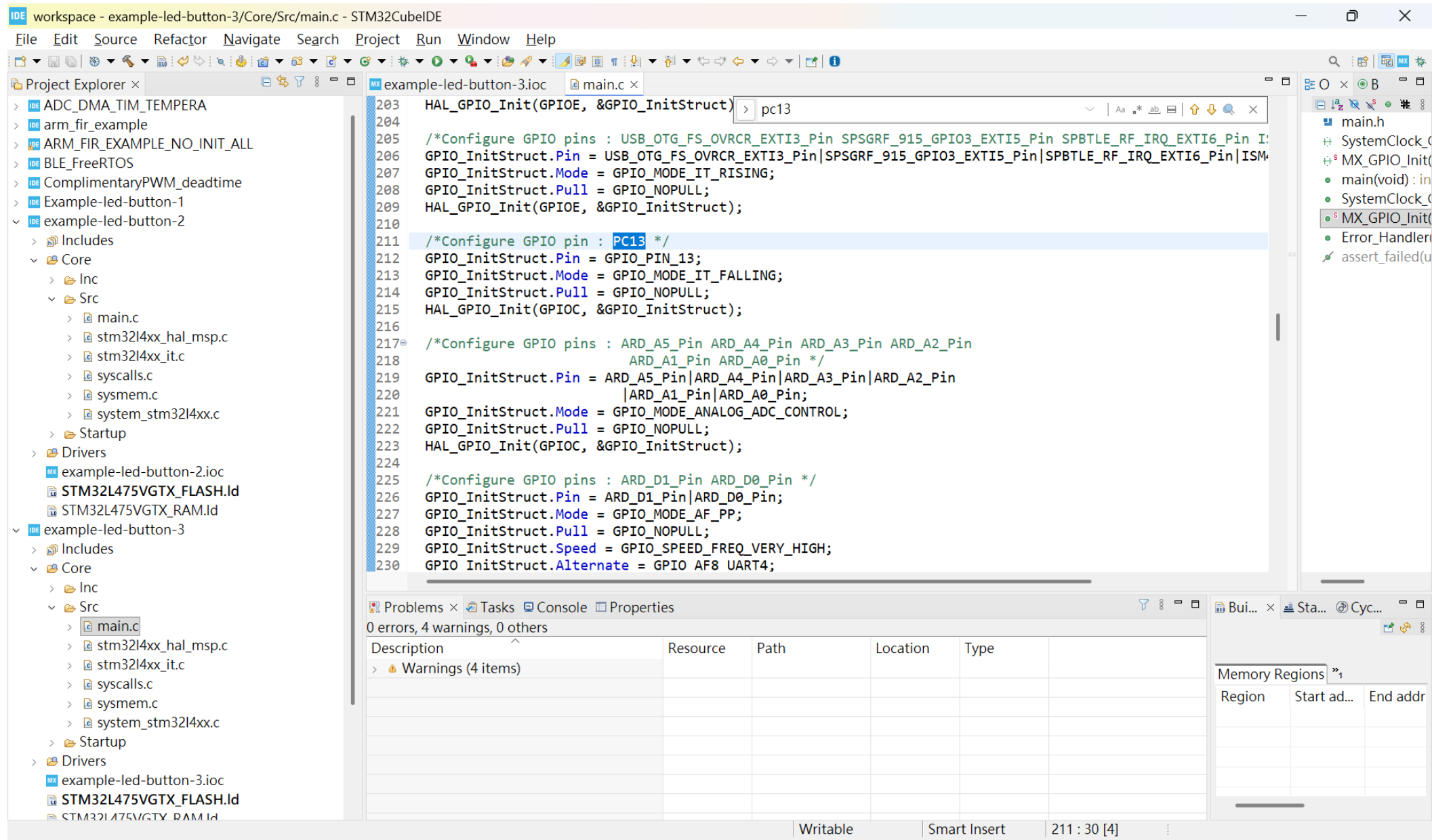
System view



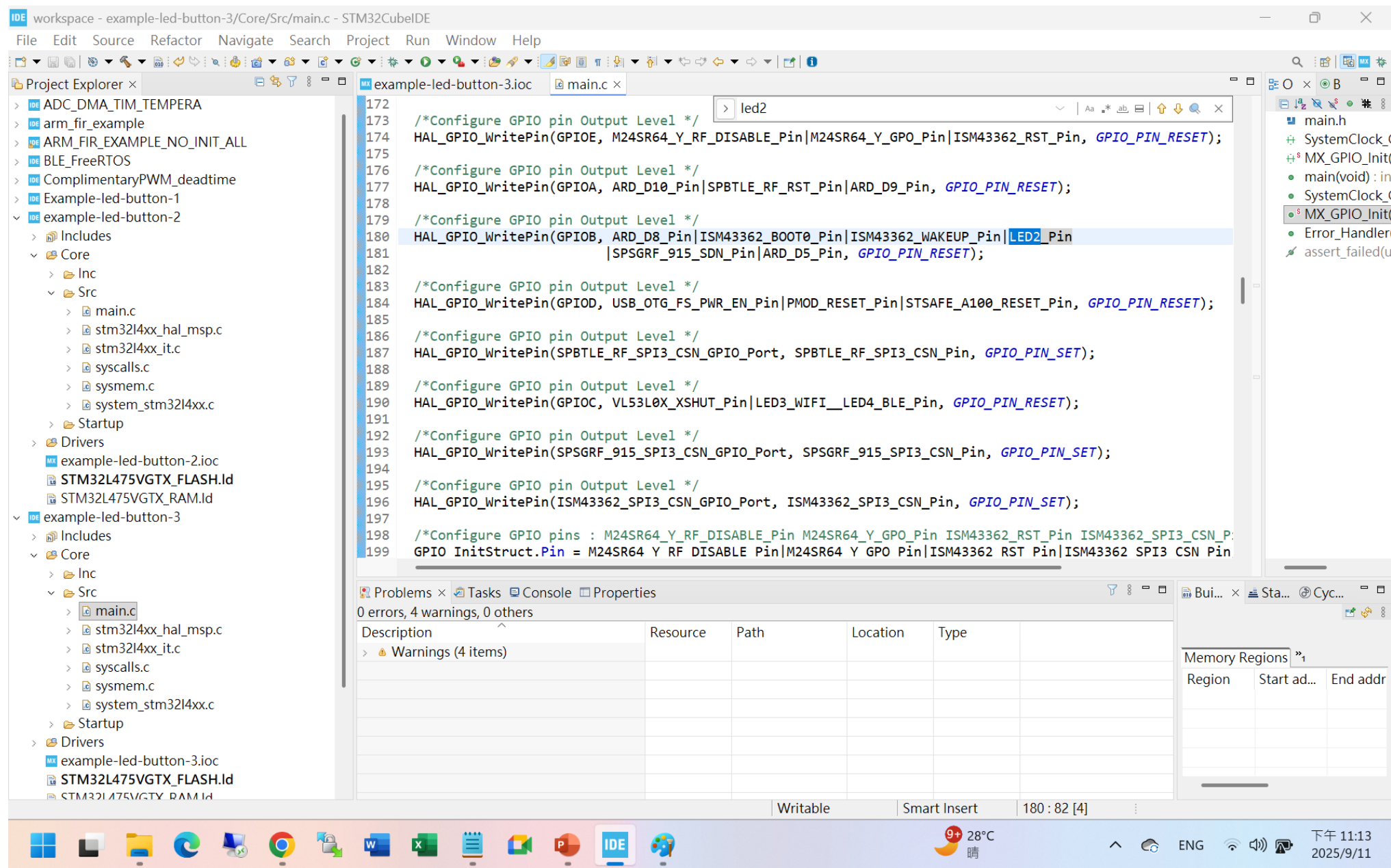
- Set PC13's GPIO mode as “External Interrupt Mode with Falling edge trigger detection”
- Generate code



- Using CubeMX to generate a template program with GPIO initial configurations (in main.c)



- LED2_Pin
 - Configure in Output Level



- Define interrupt callback function for the USER Button

IRQ Handler defined in
 ".\Drivers\STM32L4xx_HAL_Driver\Src\stm32l4xx_hal_gpio.c"

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    /* EXTI line interrupt detected */
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
    {
        HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Callback(GPIO_Pin);
    }
}
```

main.c: Define HAL_GPIO_EXTI_Callback at block USER CODE BEGIN/END 4 by entering the following codes:

```
/**
 * @brief EXTI line detection callback.
 * @param GPIO_Pin Specifies the port pin connected to corresponding EXTI line.
 * @retval None
 */
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    /* Prevent unused argument(s) compilation warning */
    UNUSED(GPIO_Pin);

    /* NOTE: This function should not be modified, when the callback is needed,
     the HAL_GPIO_EXTI_Callback could be implemented in the user file
    */
}
```

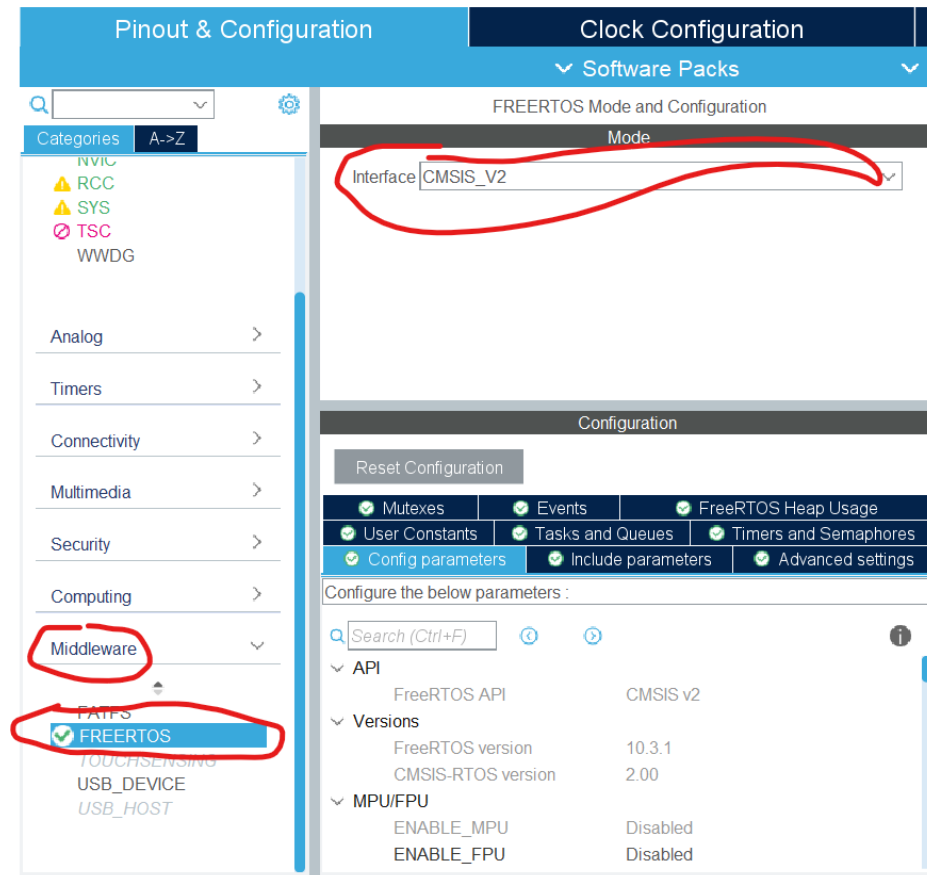
HAL_GPIO_EXTI_Callback is a weak function and can be overloaded with a user defined function.

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    switch (GPIO_Pin)
    {
        case BUTTON_EXTI13_Pin GPIO_PIN_13:
            HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
            break;
        default:
            break;
    }
}
/* USER CODE END 4 */
```

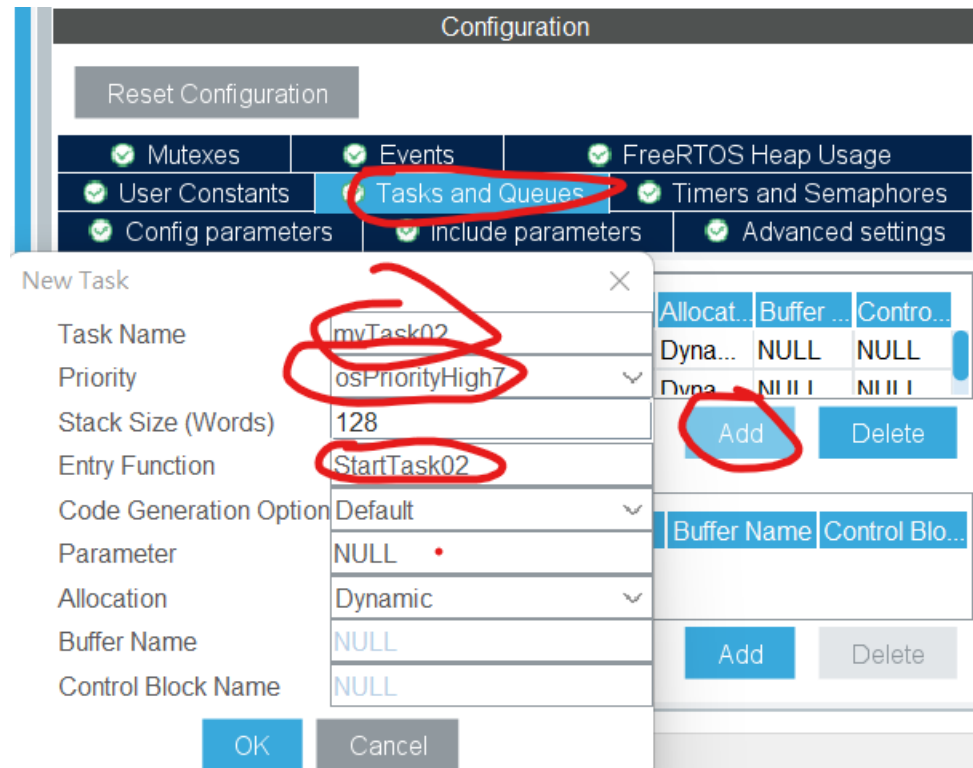
- Click "run" to download and execute the project
 - Press USER Button to see LED blinking one time
- to further test :
 - You can set PC13's GPIO mode as "Rising/Falling edge trigger detection"
 - Build and Run the project again

Example 2: use FreeRTOS threads – by defining two tasks

- Set middleware and choose FreeRTOS



- Define myTask2 with higher priority



- Code generation by CubeIDE for two FreeRTOS tasks

The priority of myTask2 is higher than defaultTask.

```
/* Definitions for defaultTask */
osThreadId_t defaultTaskHandle;
const osThreadAttr_t defaultTask_attributes = {
    .name = "defaultTask",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityNormal,
};
/* Definitions for myTask02 */
osThreadId_t myTask02Handle;
const osThreadAttr_t myTask02_attributes = {
    .name = "myTask02",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityHigh7,
};
```

Start task threads

```
/* Create the thread(s) */
/* creation of defaultTask */
defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);

/* creation of myTask02 */
myTask02Handle = osThreadNew(StartTask02, NULL, &myTask02_attributes);
```

- Define functions defaultTask() and myTask2()

```

/* USER CODE BEGIN Header_StartDefaultTask */
/**
 * @brief Function implementing the defaultTask thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_StartDefaultTask */
void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        HAL_GPIO_TogglePin(GPIOC, LED3_WIFI__LED4_BLE_Pin);
        osDelay(1000); //HAL_Delay(1000);
    }
    /* USER CODE END 5 */
}

```

```

/* USER CODE BEGIN Header_StartTask02 */
/**
 * @brief Function implementing the myTask02 thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_StartTask02 */
void StartTask02(void *argument)
{
    /* USER CODE BEGIN StartTask02 */
    /* Infinite loop */
    for(;;)
    {
        HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
        osDelay(1000); //HAL_Delay(1000);
    }
    /* USER CODE END StartTask02 */
}

```

defaultTask and myTask2 toggle LED3 and LED2, respectively.

- Build and run the project
 - Two LEDs are toggled by the two tasks, respectively.
- Questions:
 1. Are the implementations of `osDelay()` and `HAL_Delay()` different?
 - Hint: check the source codes of `osDelay()` and `HAL_Delay()`
 2. How to make two LEDs flash alternately by pressing the USER button?
 - Hint: use a semaphore and a shared variable
 - `osSemaphoreAcquire()`
 - `osSemaphoteRelease()`